

P2400R2

Library Evolution Report:

2021-06-01 to 2021-09-20

Published Proposal, 2021-09-28

Authors:

[Bryce Adelstein LeBach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)
[Fabio Fracassi — Library Evolution Vice Chair](#) (CODE University of Applied Sciences)
[Ben Craig — Library Evolution Vice Chair](#) (NI)
[Billy Baker — Library Incubator Chair](#) (FlightSafety)
[Nevin Liber — Library Incubator Vice Chair](#) (Argonne National Laboratory)
[Inbal Levi — Ranges Chair \(SG9\) and Library Mailing List Review Manager](#)
[Corentin Jabot — Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

1 Execution

- 1.1 C++23 Schedule
- 1.2 Telecons
- 1.3 Mailing List Reviews
- 1.4 Paper Queues and Backlog
- 1.5 Electronic Polling
 - 1.5.1 Expedited Advancement to Electronic Polling
- 1.6 Prioritization
- 1.7 Chair Guide
- 1.8 Staff

2 Technical

- 2.1 Plenary Approved Priorities
 - 2.1.1 Executors
 - 2.1.2 Networking
 - 2.1.3 Coroutines Library Support
 - 2.1.4 Standard Library Modules
- 2.2 Stability
 - 2.2.1 Backports
- 2.3 Other Highlights
 - 2.3.1 Ranges
 - 2.3.2 Formatting and Printing
 - 2.3.3 Text and Unicode
 - 2.3.4 `constexpr`ification
 - 2.3.5 Linear Algebra and `mdspan`
- 2.4 Papers Reviewed

This report summarizes the activities of C++'s Library Evolution group from 2021-06-01 to 2021-09-20. It is split into two sections; one focused on *execution* (process, logistics, and operations) and one focused on *technical* work (papers we processed, etc).

Readers are encouraged to also look at:

- [P2145R1](#): Evolving C++ Remotely
- [P2138R3](#): Rules of Design $\Leftrightarrow$ Specification Engagement
- [P2195R2](#): Electronic Straw Polls

- [P0592R4](#): Plan and Priorities for C++23

Prior Library Evolution reports:

- [2020-04-06 to 2020-10-22](#)
- [2020-10-27 to 2021-02-16](#)
- [2020-02-23 to 2021-05-25](#)

§ 1. Execution

§ 1.1. C++23 Schedule

We are now approaching the end of the C++23 design cycle.

As per [P1000R4](#) (C++ IS Schedule), C++23 needs to be feature complete by the 2022-02 plenary meeting. Any papers targeting C++23 will have to be advanced to electronic polling by the end of 2021, so that we can conduct a final electronic polling period.

C++23 will be the first C++ design cycle developed entirely under [our remote procedures](#).

§ 1.2. Telecons

Since 2020-04, Library Evolution has held a 90 minute telecon twice a week to review papers. Typically, 1 or 2 papers are reviewed at each telecon.

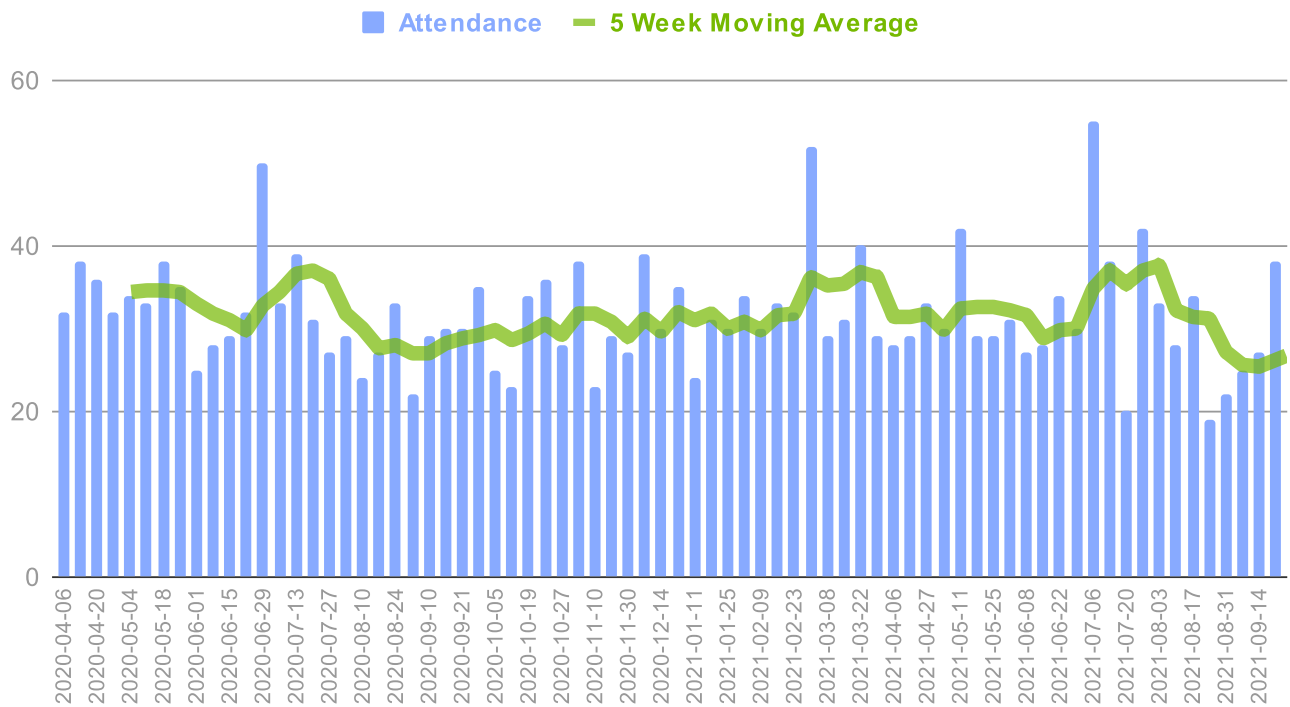
Library Evolution is planning on holding additional telecons in 2021-10 to help complete C++23. These extra telecons will be held over the course of two days and will be between 3 and 4 hours.

The agenda for upcoming telecon reviews can be found [here](#). For details on how to participate in telecon reviews, please view the calendar on [documents.isocpp.org](#) or [contact Bryce Adelstein Lelbach](#).

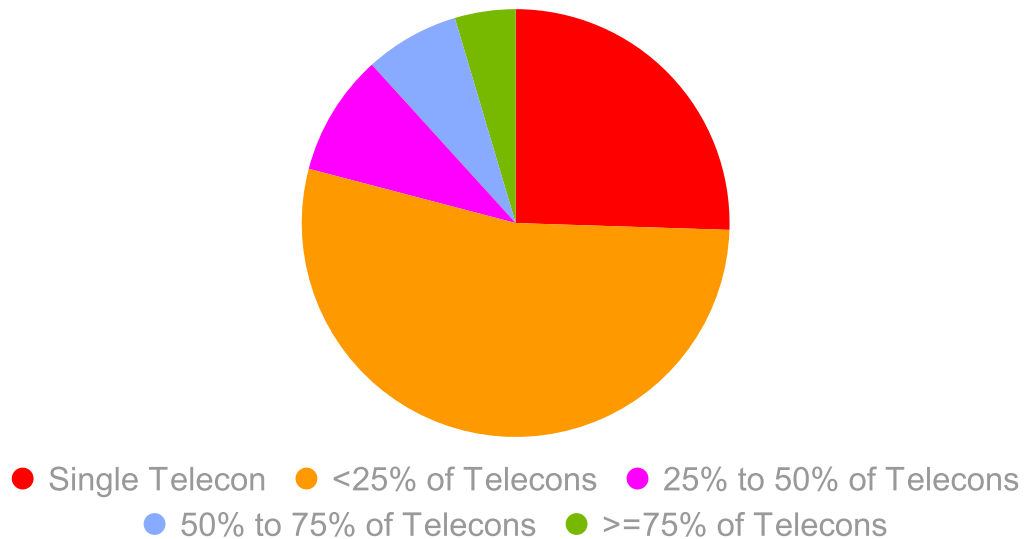
More details on how we use telecons can be found in [P2145R1](#).

	2021-06-01 to 2021-09-20	Since 2020-04-06
# of Telecons	17	74
# of Papers Reviewed On Telecons	21	107
Total # of Attendees	119	196
Mean Attendees Per Telecon	31.24	31.68
Median Attendees Per Telecon	30	30.5
Mean Telecons Per Attendee	4.46	4.5
Median Telecons Per Attendee	3	4.5

Library Evolution Telecon Attendance



Library Evolution Telecon Participation Per Attendee



§ 1.3. Mailing List Reviews

In addition to telecons, we review some papers via mailing list discussions.

Mailing list reviews are critical to our operations, as they allow us to increase our paper-processing bandwidth without increasing the frequency or duration of telecons. Additionally, certain classes of papers tend to be

particularly well suited for mailing list reviews, such as new proposals which tend need feedback, not decision making.

We conduct 2 to 3 mailing list reviews concurrently, and each review lasts for a few weeks.

The agenda for upcoming mailing list reviews can be found [here](#). For details on how to participate in mailing list reviews, please [contact Bryce Adelstein Lelbach](#).

More details on how we use mailing lists can be found in [P2145R1](#).

	2021-06-01 to 2021-09-20	Since 2020-04-06
# of Papers Reviewed On The Mailing List	12	35

§ 1.4. Paper Queues and Backlog

Queue	# of Papers
Active	41
Ready for Triage	0
Deferred	9
Ready for Mailing List Review	8
Scheduled for Mailing List Review	8
Ready for Meeting Review	14
Scheduled for Meeting Review	9
Expedited Electronic Poll Candidates	3
Ready for Electronic Poll	1
Scheduled for Electronic Poll	0

The above information is as of the publication of this paper. For up to date counts and more details on the various queues of papers being handled by Library Evolution, please see [the paper queries GitHub wiki page](#).

§ 1.5. Electronic Polling

Library Evolution typically conducts a quarterly series of electronic polls to affirm the decisions we make. Until 2022-02, Library Evolution will instead coconduct electronic polls every month as we work towards completion of C++23.

The following polling periods have been conducted over the past year:

- 2020 Fall - completed, outcomes can be found in [P2262R0](#).
- 2021 Winter - completed, outcomes can be found in [P2368R0](#).
- 2021 Spring - completed, outcomes can be found in [P2384R1](#).

- 2021 Summer - completed, outcomes can be found in [P2435R1](#).
- 2021 September - completed, outcomes can be found in [P2451R0](#).

More details on electronic polling can be found in [P2195R2](#).

§ 1.5.1. Expedited Advancement to Electronic Polling

Library Evolution telecons are the best way for us to review papers remotely, but our telecon time is limited. We started mailing list reviews of papers to reduce the demands on Library Evolution telecon time, but until recently, all papers have still required some telecon review before they are advanced to electronic polls and out of Library Evolution.

In the last few months, we have begun experimenting with advancing papers directly from mailing list review to electronic polling. The procedure is intended for smaller matters, non-controversial matters, and matters that we believe will not benefit from telecon discussion. It works as follows:

0.) A mailing list thread is started on lib-ext@ proposing that a paper be advanced directly to electronic polling. Background on the paper, its maturity, and the outcomes of previous discussions is provided. 1.) Participants are asked to take a stance on the motion by replying with either a "+1" or a "-1". 2.) When the thread reaches its nature conclusion, the chairs determine whether it is suitable for the paper to proceed to electronic polling. Typically, we are looking for strong and vocal support; a lack of objections is not sufficient.

§ 1.6. Prioritization

In conjunction with Library, we've prioritized all the papers that have been forwarded to Library, using the priority levels identified in [P0592R4](#) (Plan and Priorities for C++23). See the [prioritized list of papers](#) for details.

§ 1.7. Chair Guide

We have developed a Library Evolution chairing guide, which can be found on the [Library Evolution GitHub wiki](#)

§ 1.8. Staff

Library Evolution is organized by a team, not a single individual:

- Bryce Adelstein Lelbach, Chair
- Fabio Fracassi, Vice Chair
- Ben Craig, Vice Chair
- Billy Baker, Incubator Chair

- Nevin Liber, Incubator Vice Chair
- Corentin Jabot, Mailing List Review Manager
- Inbal Levi, Mailing List Review Manager and Algorithms Task Force Chair

Leadership by a group instead of a team has a number of benefits:

- Increases our capacity and capabilities.
- Reduces the burden on each leader.
- Builds institutional knowledge and redundancy.

We hold a regular staff meeting every two weeks to discuss our operations and make plans for the future.

Additionally, the following noble individuals take minutes for Library Evolution, and deserve accolades:

- Ben Craig
- Inbal Levi
- Mark Hoemmen
- Guy Davidson
- Dietmar Kühl

§ 2. Technical

§ 2.1. Plenary Approved Priorities

A plenary of the C++ Committee approved [P0592R4](#) (Plan and Priorities for C++23). This section describes the status of the priority Standard Library features described in that plan.

Plenary Approved Priority	Status
Executors	Review scheduled. At risk for C++23.
Networking	Review scheduled. At risk for C++23.
Coroutines Library Support	Waiting for revisions. At risk for C++23; active authors and champions needed.
Standard Library Modules	Review scheduled. On track for C++23.

§ 2.1.1. Executors

Over the Summer of 2021, Library Evolution reviewed [P2300R0](#) (`std::execution`), a new streamlined Executors proposal from some of the authors of [P0443R14](#) (A Unified Executors Proposal for C++). The aim of P2300 was to simplify Executors down to a minimal core and make the proposal more accessible and easier to

understand. The new proposal was well received by Library Evolution, and we had consensus that we want to proceed with P2300 instead of P0443.

However, there is still disagreement between the proponents of P2300 and the proponents of [P2444R0](#), the async model of the Networking TS. During our discussions, we have begun to question whether we need to pursue a single unified asynchronous model for both the Networking TS and structured concurrency.

It is unclear at this point whether P2300, or any other form of Executors, will be ready for C++23. While we have consensus on the design of P2300, there are still questions about field experience and maturity. Given that the window for C++23 closes at the end of 2021, the likelihood of including Executors in C++23 is rapidly diminishing.

§ 2.1.2. Networking

As part of our discussion of Executors and its relationship to the Networking TS, some have suggested that we should consider shipping the Networking TS as-is in C++23. This will be given due consideration and a decision will be made by 2021-10.

§ 2.1.3. Coroutines Library Support

A revision of [P2168R1](#) (`std::generator`) was recently received and reviewed multiple times in 2021-02, 2021-03, and 2021-04. The authors have been given thorough feedback and will return with a revision in the coming months.

Otherwise, no further work on the Coroutines Library Support has occurred since Spring 2020 due to a lack of new papers and a lack of revisions of existing papers.

Some of the other work on Coroutines Library Support depends on Executors and may be blocked for now.

§ 2.1.4. Standard Library Modules

In 2021-09, we reviewed [P2412R0](#) (Minimal Modules for the Standard Library), which proposes that we add one big `std` module for C++23. Library Evolution was in favor of this plan. The paper should return for another round of review in 2021-10, at which point it will hopefully be advanced to electronic polling for C++23.

We also plan to review the latest revision of [P0581R2](#) (Standard Library Modules), a more future looking proposal that suggests a fine grained set of standard library modules in addition to one big `std` module.

§ 2.2. Stability

§ 2.2.1. Backports

Typically, it is very difficult or impossible to make breaking changes to Standard C++ facilities after they have shipped in a particular C++ Standard. However, sometimes there is a small window of opportunity to make such changes.

If no implementation has shipped a particular Standard C++ facility in production, then no one is using the facility yet. Therefore, it is possible for the C++ committee to make what would be a breaking change to said facility in the next C++ Standard without actually causing any breakage. This is only possible if all vendors agree to accept the change and apply it as a backport to their implementation of the older C++ Standard.

Backporting breaking changes to Standard C++ facilities in this way is an extraordinary measure and must only be done with the utmost of care. It should not be considered the norm and we should not assume that we will be able to do it in the future.

When we make these changes, we are effectively changing a C++ Standard after it has shipped. This can cause a great deal of uncertainty in the ecosystem for both users and vendors, because it makes it unclear when a C++ Standard is complete and stable. It may even slow down adoption of C++20. As such, this is a tool we must use very delicately.

Over the past few months, Library Evolution has taken advantage of this window to fix a number of issues, mostly relating to ranges and text formatting in C++20:

- [P2372R0](#): Fixing locale handling in chrono formatters
- [P2216R3](#): format improvements
- [P2328R1](#): join_view should join all views of ranges
- [P2325R3](#): Views should not be required to be default constructible
- [P2210R2](#): Superior string splitting
- [P2367R0](#): Remove misuses of list-initialization from Clause 24
- [P2281R1](#): Clarifying range adaptor objects
- [P2231R1](#): Missing constexpr in optional and variant
- [P2166R1](#): Prohibit basic_string and basic_string_view construction from nullptr

We expect the following three C++20 backports to be moved at the 2021-10 plenary meeting:

- [P2418R2](#): Add support for std::generator-like types to std::format
- [P2415R2](#): What is a view?
- [P2432R1](#): Fix istream_view

Library Evolution does not expect to consider any additional backports for C++20.

The above information is as of the publication of this paper. For up to date information and more details on Library backports [the backports GitHub wikipage](#).

§ 2.3. Other Highlights

§ 2.3.1. Ranges

We are making significant progress on the objectives laid out in [P2214R0](#) (A Plan for C++23 Ranges). We expect to deliver at least all of the tier 1 components of P2214R0.

The following ranges proposals were advanced to Library Working Group for C++23 by electronic poll:

- [P1206R6](#): `ranges::to`
- [P2415R1](#): What Is A view?
- [P2432R0](#): Fix `istream_view`

Over the past few months, we have reviewed the following ranges proposals at Library Evolution telecons:

- [P1206R6](#): Conversions from ranges to containers
- [P1664R3](#): `ranges::reconstructible_range`
- [P2138R4](#): Rules of Design $\Leftrightarrow$ Specification Engagement
- [P2276R1](#): Fix `std::cbegin()`, `std::ranges::cbegin`, and `cbegin()` for span
- [P2374R1](#): `views::cartesian_product`

The following ranges papers were reviewed at Ranges Study Group (SG9) telecons:

- [P2408R0](#): Ranges views as inputs to non-Ranges algorithms
- [LWG3564](#): `transform_view::iterator::value_type` and `iterator_category` should use `const F&`
- [P2165R2](#): Compatibility between `tuple`, `pair` and tuple-like objects
- [P2387R0](#): Pipe support for user-defined range adaptors
- [P1664R5](#): `reconstructible_range` - a concept for putting ranges back together
- [LWG3564](#): transform function constness
- [LWG3534](#): `ranges::set_intersection` and `ranges::set_difference` algorithm requirements are too strict

§ 2.3.2. Formatting and Printing

[P2093R5](#) (Formatted Output) continues to mature. The Text and Unicode group (SG16) has sent it back to Library Evolution for what should be a final review for C++23 in 2021-11.

We've made progress on [P2286R1](#) (Formatting Ranges) as well, reviewing it at a Library Evolution telecon. We expect it to advance in time for C++23.

We also reviewed and advanced [P2418R0](#) (Add Support For `std::generator`-like Types To `std::format`) to Library Working Group for C++23 and as a backport for C++20.

§ 2.3.3. Text and Unicode

We've nearly completed our work on [P1885R5](#) (Naming Text Encodings to Demystify Them), which is a part of the roadmap laid out in [P1238R0](#) (Unicode Direction) and one of their priorities for C++23. We are planning to take another look at it in the next few months and then hopefully advance it for C++23.

§ 2.3.4. constexprification

Over the summer, we advanced [P0533R8](#) (constexpr For `<cmath>` And `<cstdlib>`) back to Library Working Group for C++23. We also reviewed [P1383R0](#) (More constexpr for `<cmath>`) and [P2337R0](#) (Less constexpr for `<cmath>`).

We also advanced the following constexprification proposals to Library Working Group by electronic poll:

- [P0533R8](#): constexpr For `<cmath>` And `<cstdlib>`
- [P2273R2](#): Making `unique_ptr` constexpr
- [P2291R2](#): Add constexpr Modifiers To Functions `to_chars` And `from_chars` For Integral Types In `<charconv>` Header

And the following papers were reviewed on the Library Evolution mailing list:

- [P2417R0](#): A more constexpr bitset

§ 2.3.5. Linear Algebra and `mdspan`

We looked at [P0009R12](#) (`mdspan`) and some papers modifying it ([P2299R3](#)) over the summer. We decided to retarget `mdspan` for C++23. We expect to review the paper again in 2021-11.

§ 2.4. Papers Reviewed

The following papers were reviewed at Library Evolution telecons and advanced to Library Working Group in the 2021 Summer Library Evolution polling period (see [P2435R1](#) for details):

- [P2372R1](#): Fixing Locale Handling In Chrono Formatters
- [P1206R6](#): `ranges::to`

- [P0533R8](#): constexpr For <cmath> And <cstdlib>
- [P2273R2](#): Making unique_ptr constexpr

The following papers were reviewed at Library Evolution telecons and advanced to Library Working Group in the 2021 September Library Evolution polling period (see [P2451R0](#) for details):

- [P2418R0](#): Add Support For std::generator-like Types To std::format
- [P2415R1](#): What Is A view?
- [P2432R0](#): Fix istream_view
- [P2351R0](#): Mark All Library Static Cast Wrappers As [[nodiscard]]
- [P2291R2](#): Add constexpr Modifiers To Functions to_chars And from_chars For Integral Types In <charconv> Header

The following papers were reviewed at Library Evolution telecons:

- [P0009R12](#): mdspan
- [P1206R6](#): Conversions from ranges to containers
- [P1383R0](#): More constexpr for <cmath>
- [P2337R0](#): Less constexpr for <cmath>
- [P1664R3](#): ranges::reconstructible_range
- [P1673R3](#): BLAS Linear Algebra
- [P1885R5](#): Naming Text Encodings to Demystify Them
- [P2138R4](#): Rules of Design <=> Specification Engagement
- [P2276R1](#): Fix std::cbegin(), std::ranges::cbegin, and cbegin() for span
- [P2286R2](#): Formatting Ranges
- [P2299R3](#): mdspans of All Dynamic Extents
- [P2300R1](#): std::execution
- [P2345R0](#): Relaxing Requirements of Moved-From Objects
- [P2372R1](#): Fixing locale handling in chrono formatters
- [P2374R1](#): views::cartesian_product
- [P2412R0](#): Minimal Module Support for the Standard Library
- [P2415R1](#): What is a view?
- [P2418R0](#): Adding support for std::generator to std::format
- [P2428R0](#): Issues and questions with P2300 std::execution
- [P2432R0](#): Fix istream_view
- [P2444R0](#): Asio Async Model

The following papers were reviewed on the Library Evolution mailing list:

- [P2332R0](#): Establishing `std::hive` as replacement name for the proposed `std::colony` container
- [P2066R7](#): Suggested draft TS for C++ Extensions for Minimal Transactional Memory
- [P1339R1](#): Disallowing the friending of names in namespace `std`
- [P2226R0](#): A proposal for an idiom to move from an object and reset it to its default constructed state
- [P2291R1](#): Add Constexpr Modifiers to Functions `to_chars` and `from_chars` for Integral Types in Header
- [P2249R0](#): Mixed comparisons for smart pointers
- [P2351R0](#): Mark all library static cast wrappers as `[[nodiscard]]`
- [P2377R0](#): `[[nodiscard]]` in the Standard Library: Clause 23 Iterators library
- [P0493R2](#): Atomic maximum/minimum
- [P2413R0](#): Remove unsafe conversions of `unique_ptr`
- [P2370R0](#): Stacktrace from exception
- [P2417R0](#): A more constexpr bitset